each link only shows for nonempty values. Remember that empty values are treated as `False` for the purposes of the `{% if %}` tag.

Listing 3.18 A pagination footer

```
{% if prev %}
    <a href="?page={{prev}}">Prev</a>
{% endif %}
{% if next %}
    <a href="?page={{next}}">Next</a>
{% endif %}
```

Check if there is a previous page.

If so, link to the current page with a new query parameter containing the page number.

The preceding subtemplate can be reused in any web page via `{% include %}`. To populate the values, the tag uses the `with` format providing numbers for `prev` and `next`. The following listing shows the use of the subtemplate for page 5.

Listing 3.19 Including a subtemplate with values

```
<h3> More Content </h3>

{% include "pagination.html" with prev=4 next=6 %}
```

Normally, the first argument to `{% include %}` is a string containing the name of the subtemplate to include. The tag also supports the use of variables to specify the subtemplate name. If you're doing something particularly dynamic, you can get the subtemplate name from the data context. That means you can include different subtemplates, based on the context set in your view.

Templates are a powerful tool to manage HTML in a manner similar to Python. Instead of writing large files with lots of repetition, you build composable parts. Instead of hardcoding content, you build segments that include conditional rendering and loops that change the page based on the context data passed in from a view. All of this results in more maintainable code. The next step is to extend this power to your views: instead of using hardcoded data dictionaries, you retrieve content from a database.

3.9 Exercises

Practice makes perfect. Here are some activities you can try on your own:

1. Create a new base HTML file using your favorite web style framework, such as Bootstrap or Tailwind. Once you have something pretty, test it out by changing news2.html to extend from it.
2. Write a more advanced version of the News page by adding a `news_advanced()` view to home/views.py. Each data item in this view is a tuple containing the date and subject of the news. Create a news_adv.html template that shows the data in a table. Look-up the `{% cycle %}` tag and the `|date` filter in appendix C, and